Agentic Summarizer — Developer Walkthrough

Author: Jibril Anifowoshe — September 2025

Setup & Requirements

Install Ollama and pull the model:

```
ollama serve
ollama pull llama3.2:3b
```

Install Python dependencies:

```
pip install -r requirements.txt
jupyter notebook
```

Notebook Cells

Cell 1 — Bootstrap + Health Check

```
#
==========================================================================
=
# Prototype built by Jibril Anifowoshe
# Agentic Summarizer using Ollama (+ LangChain @tool wrapper)
# Model: llama3.2:3b
# September 2025
#
# CELL 1 — Bootstrap + Health Check
# - Installs deps into THIS Jupyter kernel
# - Imports core libs (+ only langchain.tools for @tool)
# - Verifies Ollama is running
#
# Terminal A:  ollama serve
# Terminal B:  ollama pull llama3.2:3b
#
==========================================================================
=

# 0) Install/verify packages
import sys, subprocess
def _pip_install(pkgs):
    subprocess.check_call([sys.executable, "-m", "pip", "install", *pkgs])

_pip_install([
    "requests",
    "beautifulsoup4",
    "readability-lxml",
```

```
    "ipywidgets",
    "reportlab",
    "langchain>=0.2",      # ONLY for the @tool decorator
])

# 1) Imports
import re
import urllib.request
import shutil
from datetime import datetime

import requests
from bs4 import BeautifulSoup
from readability import Document
from langchain.tools import tool   # <-- the only LangChain import we keep

# 2) Ollama health check
def _ollama_running(url: str = "http://127.0.0.1:11434/api/tags", timeout: int
= 3) -> bool:
    try:
        with urllib.request.urlopen(url, timeout=timeout) as r:
            return r.status == 200
    except Exception:
        return False

# 3) Guidance
if not shutil.which("ollama"):
    print("❌ Ollama CLI not found.\n"
          "   macOS: brew install ollama\n"
          "   Windows: winget install Ollama.Ollama\n"
          "Then in a terminal:  ollama serve")
elif not _ollama_running():
    print("⚠️ Ollama is installed but not running.\n"
          "   Start it in a terminal:  ollama serve\n"
          "   Then re-run this cell.")
else:
    print("✅ Ollama is reachable at http://127.0.0.1:11434")
    print("✅ Packages ready. Proceed to Cell 2 (Tools).")
```

Cell 2 — Tools (@tool wrapper)

```
#
================================================================================
=
# CELL 2 — Tools the agent can call (via .invoke)
# - fetch_url(url):  fetch + clean article text
# - save_note(filename, content): save .md with signature footer
#
================================================================================
=

@tool
def fetch_url(url: str) -> str:
    """
```

```
    Fetch a web page and return the cleaned article text (not the full HTML).
    Steps:
      - HTTP GET with a standard User-Agent
      - 'readability' to isolate main content
      - Plaintext via BeautifulSoup
      - Whitespace normalization
      - Trim to a safe size for the model
    """
    try:
        headers = {"User-Agent": "Mozilla/5.0"}
        html = requests.get(url, headers=headers, timeout=20).text
        article_html = Document(html).summary(html_partial=True)
        text = BeautifulSoup(article_html, "html.parser").get_text("\n")
        text = re.sub(r"\n{3,}", "\n\n", text).strip()
        return text[:20000]
    except Exception as e:
        return f"[fetch_url error] {e}"

@tool
def save_note(filename: str, content: str) -> str:
    """
    Save content to a Markdown (.md) file (append '.md' if missing).
    Appends a signature footer (credit).
    """
    try:
        if not filename.endswith(".md"):
            filename += ".md"
        footer = "\n\n---\nSummary generated with Agentic Summarizer prototype
— Jibril Anifowoshe"
        with open(filename, "w", encoding="utf-8") as f:
            f.write(content + footer)
        return f"Saved notes to {filename}"
    except Exception as e:
        return f"[save_note error] {e}"

TOOLS = [fetch_url, save_note]
print("✅ Tools ready. Proceed to Cell 3 (Summarizer).")
```

Cell 3 — Summarizer (Direct Ollama)

```
#
==========================================================================
=
# Cell 3 — Summarizer (direct Ollama; faster model + safe stops)
# Model: 'llama3.2:3b'  (pull it first:  ollama pull llama3.2:3b)
# - Single URL: 2800 chars
# - 3 URLs: 2000 chars per page (Cell 4 passes this value)
# - Output limited to 5 bullets via stop tokens
#
==========================================================================
=

import requests
```

```
MODEL = "llama3.2:3b"
OLLAMA_URL = "http://127.0.0.1:11434/api/generate"

SYSTEM = """You are a concise research agent.
Return EXACTLY 5 bullets as a NUMBERED LIST:
1. ...
2. ...
3. ...
4. ...
5. ...
- Executive tone, no preamble or closing lines.
- No quotes, no code fences, no extra bullets.
"""

def _ollama_generate(prompt: str,
                     model: str = MODEL,
                     num_predict: int = 240,
                     num_ctx: int = 2048,
                     timeout_s: int = 35) -> str:
    """Call Ollama's /api/generate once (non-stream) and return the text."""
    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False,
        "options": {
            "num_predict": num_predict,
            "num_ctx": num_ctx,
            "temperature": 0,
            "stop": ["\n6.", "\n## ", "\n### "]  # stop after 5 bullets
        }
    }
    r = requests.post(OLLAMA_URL, json=payload, timeout=timeout_s)
    r.raise_for_status()
    return r.json().get("response", "")

def summarize_text(article_text: str, max_chars: int = 2800) -> str:
    """Return EXACTLY 5 numbered bullets for the given article text."""
    txt = (article_text or "")[:max_chars]
    prompt = f"{SYSTEM}\n\nSummarize into EXACTLY 5 numbered bullets (1-
5):\n\n{txt}"
    return _ollama_generate(prompt)

print(f"✅ Summarizer ready using model: {MODEL}")
```

Cell 4 — Tabbed UI

```
#
================================================================================
=
# CELL 4 — Tabbed UI (Single URL + 3 URLs + Compare) — direct Ollama path
# - Single URL uses 2,800 chars; 3-URL batch uses 2,000 chars each (fast)
# - Comparison uses the same direct _ollama_generate() as Cell 3 (no LangChain)
# - Formatter guarantees EXACTLY 5 numbered bullets
#
```

```
================================================================================
=

import ipywidgets as w
from IPython.display import display
from urllib.parse import urlparse
import re
from datetime import datetime

# --------- Helpers ------------------------------------------------------------
---
def _smart_filename_from_url(url: str, fallback: str = "summary") -> str:
    try:
        host = urlparse(url).netloc or ""
        host = re.sub(r"^www\.", "", host)
        host = re.sub(r"[^a-zA-Z0-9_-]+", "_", host).strip("_")
        date = datetime.now().date().isoformat()
        return f"{host or fallback}_{date}"
    except Exception:
        return fallback

def _normalize_basename(name: str) -> str:
    name = (name or "").strip().lstrip(". ").strip()
    for ext in (".md", ".pdf", ".txt", ".markdown"):
        if name.lower().endswith(ext):
            name = name[: -len(ext)]
            break
    return name

def _format_numbered_five(raw: str, url: str) -> str:
    """Normalize output: title + EXACTLY 5 numbered bullets (1-5)."""
    title = f"### Here's the summary of {url}\n"
    text = (raw or "").strip()

    # Prefer real numbered bullets from the model if present
    pts = re.split(r"(?m)^\s*(\d{1,2})\.\s+", text)
    numbered = []
    if len(pts) > 1:
        for i in range(1, len(pts) - 1, 2):
            body = pts[i+1].strip()
            if body:
                numbered.append(body)
        numbered = numbered[:5]

    # Fallback: clean lines and re-number
    if not numbered:
        lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
        cleaned = []
        for ln in lines:
            ln = re.sub(r"^\s*[-•\d]+\s*[.)-]?\s*", "", ln)
            if ln:
                cleaned.append(ln)
        numbered = cleaned[:5]

    while len(numbered) < 5:
```

```
        numbered.append("…")

    bullets = "\n".join(f"{i}. {p}" for i, p in enumerate(numbered, 1))
    return f"{title}\n{bullets}"

# --------- PDF helper ------------------------------------------------------
---
try:
    _ensure_reportlab
except NameError:
    def _ensure_reportlab():
        try:
            import reportlab  # noqa
            return True
        except Exception:
            import sys, subprocess
            try:
                subprocess.check_call([sys.executable, "-m", "pip", "install",
"reportlab"])
                import reportlab  # noqa
                return True
            except Exception:
                return False

try:
    _save_pdf
except NameError:
    def _save_pdf(path: str, content_text: str) -> str:
        from reportlab.lib.pagesizes import LETTER
        from reportlab.pdfgen import canvas
        from reportlab.lib.units import inch
        c = canvas.Canvas(path, pagesize=LETTER)
        width, height = LETTER
        x, y = 1*inch, height - 1*inch
        c.setFont("Helvetica", 11)
        for line in content_text.splitlines():
            while len(line) > 100:
                c.drawString(x, y, line[:100]); line = line[100:]
                y -= 0.22*inch
                if y < 1*inch:
                    c.showPage(); y = height - 1*inch; c.setFont("Helvetica",
11)
            c.drawString(x, y, line)
            y -= 0.22*inch
            if y < 1*inch:
                c.showPage(); y = height - 1*inch; c.setFont("Helvetica", 11)
        c.setFont("Helvetica-Oblique", 9)
        c.drawString(x, y, "— Generated by Agentic Summarizer prototype —
Jibril Anifowoshe")
        c.save()
        return f"Saved PDF to {path}"

# ============================== Single URL tab
==================================
s_url = w.Text(
```

```
    value="https://en.wikipedia.org/wiki/Zero_trust_architecture",
    description="URL:",
    layout=w.Layout(width="95%")
)
s_name = w.Text(
    value="",
    description="Save as:",
    placeholder="optional (auto if empty)",
    layout=w.Layout(width="60%")
)
s_md = w.Checkbox(value=True, description="Save .md")
s_pdf = w.Checkbox(value=False, description="Save .pdf")
s_go = w.Button(description="Summarize", button_style="primary")
s_out = w.Output(layout=w.Layout(border="1px solid #ddd", padding="8px"))

def _single_go(_):
    s_out.clear_output()
    url = s_url.value.strip()
    if not url or not re.match(r"^https?://", url):
        with s_out: print("❌ Enter a valid URL (http/https)."); return
    base = _normalize_basename(s_name.value.strip()) or
_smart_filename_from_url(url)

    with s_out: print(f"Task: Summarize {url}")

    # Fetch
    article = fetch_url.invoke({"url": url})
    if isinstance(article, str) and article.startswith("[fetch_url error]"):
        with s_out: print(article); return

    # Summarize (single URL gets 2,800 chars)
    raw = summarize_text(article, max_chars=2800)
    content_text = _format_numbered_five(raw, url)

    # Save
    saved = []
    if s_md.value:
        saved.append(save_note.invoke({"filename": base, "content":
content_text}))
    if s_pdf.value and _ensure_reportlab():
        saved.append(_save_pdf(f"{base}.pdf", content_text))

    with s_out:
        print("\n---- RESULT ----\n")
        print(content_text)
        print("\n— Generated by Agentic Summarizer prototype — Jibril
Anifowoshe")
        if saved:
            print("\n" + "\n".join(saved))
            print("(Files saved next to this notebook.)")

s_go.on_click(_single_go)
single_tab = w.VBox([s_url, w.HBox([s_name, s_md, s_pdf, s_go]), s_out])

# ============================ 3 URLs + Compare tab
```

```
================================
b_u1 = w.Text(value="https://en.wikipedia.org/wiki/Zero_trust_architecture",
description="URL A:", layout=w.Layout(width="95%"))
b_u2 = w.Text(value="https://en.wikipedia.org/wiki/Zero_trust_security_model",
description="URL B:", layout=w.Layout(width="95%"))
b_u3 =
w.Text(value="https://en.wikipedia.org/wiki/Defense_in_depth_(computing)",
description="URL C:", layout=w.Layout(width="95%"))
b_name = w.Text(value="", description="Save as:", placeholder="optional (auto
if empty)", layout=w.Layout(width="60%"))
b_md = w.Checkbox(value=True, description="Save combined .md")
b_pdf = w.Checkbox(value=False, description="Save combined .pdf")
b_go = w.Button(description="Summarize & Compare", button_style="primary")
b_out = w.Output(layout=w.Layout(border="1px solid #ddd", padding="8px"))

def _compare_block(s1: str, s2: str, s3: str) -> str:
    """Compare three summaries via the same direct Ollama call (fast +
bounded)."""
    prompt = (
        "You are a concise analyst. Compare the three summaries provided.\n"
        "Output exactly this structure:\n"
        "Common themes:\n1. ...\n2. ...\n3. ...\n"
        "Key differences:\n1. ...\n2. ...\n3. ...\n"
        "Standout insight per source:\nA) ...\nB) ...\nC) ...\n"
        "Takeaway:\nOne sentence. Be crisp and non-redundant.\n\n"
        f"Summary A:\n{s1}\n\nSummary B:\n{s2}\n\nSummary C:\n{s3}\n"
    )
    return _ollama_generate(
        prompt,
        model=MODEL,
        num_predict=220,
        num_ctx=2048
    )

def _batch_go(_):
    b_out.clear_output()
    urls = [b_u1.value.strip(), b_u2.value.strip(), b_u3.value.strip()]
    if any(not u or not re.match(r"^https?://", u) for u in urls):
        with b_out: print("❌ All three URLs must start with http/https.");
return
    base = _normalize_basename(b_name.value.strip()) or
_smart_filename_from_url(urls[0], "summary_batch")

    with b_out: print("Task: Summarize 3 URLs and produce a comparison.")

    blocks = []
    for i, url in enumerate(urls, 1):
        with b_out: print(f"\nFetching article {i}…")
        article = fetch_url.invoke({"url": url})
        if isinstance(article, str) and article.startswith("[fetch_url
error]"):
            with b_out: print(article); return

        with b_out: print(f"Summarizing {i}…")
        raw = summarize_text(article, max_chars=2000)  # 3-URL batch stays lean
```

```
            formatted = _format_numbered_five(raw, url)

            if len(formatted) > 2000:
                formatted = formatted[:2000]

            blocks.append(formatted)

        with b_out: print("\nComparing A vs B vs C…")
        comp = _compare_block(blocks[0], blocks[1], blocks[2])

        combined = []
        combined.append("# Multi-URL Summaries and Comparison\n")
        for lbl, blk in zip(["A", "B", "C"], blocks):
            combined.append(f"## Summary {lbl}\n")
            combined.append(blk)
            combined.append("")
        combined.append("## Comparison")
        combined.append(comp)
        combined.append("\n— Generated by Agentic Summarizer prototype — Jibril
Anifowoshe\n")
        text_combined = "\n".join(combined)

        with b_out:
            print("\n---- RESULT ----\n")
            print(text_combined)

        saved = []
        if b_md.value:
            saved.append(save_note.invoke({"filename": base + "_comparison",
"content": text_combined}))
        if b_pdf.value and _ensure_reportlab():
            saved.append(_save_pdf(f"{base}_comparison.pdf", text_combined))
        if saved:
            with b_out:
                print("\n" + "\n".join(saved))
                print("(Files saved next to this notebook.)")

b_go.on_click(_batch_go)
compare_tab = w.VBox([b_u1, b_u2, b_u3, w.HBox([b_name, b_md, b_pdf, b_go]),
b_out])

# --- Tabs ----------------------------------------------------------------
---
tabs = w.Tab(children=[single_tab, compare_tab])
tabs.set_title(0, "Single URL")
tabs.set_title(1, "3 URLs + Compare")
display(tabs)
```

src/ Modules

```
import re
import requests
from bs4 import BeautifulSoup
```

```
from readability import Document

def fetch_url(url: str) -> str:
    """Fetch a page and return cleaned article text."""
    try:
        headers = {"User-Agent": "Mozilla/5.0"}
        html = requests.get(url, headers=headers, timeout=20).text
        article_html = Document(html).summary(html_partial=True)
        text = BeautifulSoup(article_html, "html.parser").get_text("\n")
        text = re.sub(r"\n{3,}", "\n\n", text).strip()
        return text[:20000]
    except Exception as e:
        return f"[fetch_url error] {e}"


def save_note(filename: str, content: str) -> str:
    """Save content to a Markdown file and append signature footer."""
    try:
        if not filename.endswith(".md"):
            filename += ".md"
        footer = "\n\n---\nSummary generated with Agentic Summarizer prototype
— Jibril Anifowoshe"
        with open(filename, "w", encoding="utf-8") as f:
            f.write(content + footer)
        return f"Saved notes to {filename}"
    except Exception as e:
        return f"[save_note error] {e}"


import requests

MODEL = "llama3.2:3b"
OLLAMA_URL = "http://127.0.0.1:11434/api/generate"

SYSTEM = """You are a concise research agent.
Return EXACTLY 5 bullets as a NUMBERED LIST:
1. ...
2. ...
3. ...
4. ...
5. ...
- Executive tone, no preamble or closing lines.
- No quotes, no code fences, no extra bullets.
"""

def _ollama_generate(prompt: str,
                     model: str = MODEL,
                     num_predict: int = 240,
                     num_ctx: int = 2048,
                     timeout_s: int = 35) -> str:
    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False,
        "options": {
```

```
            "num_predict": num_predict,
            "num_ctx": num_ctx,
            "temperature": 0,
            "stop": ["\n6.", "\n## ", "\n### "]
        }
    }
    r = requests.post(OLLAMA_URL, json=payload, timeout=timeout_s)
    r.raise_for_status()
    return r.json().get("response", "")

def summarize_text(article_text: str, max_chars: int = 2800) -> str:
    txt = (article_text or "")[:max_chars]
    prompt = f"{SYSTEM}\n\nSummarize into EXACTLY 5 numbered bullets (1-
5):\n\n{txt}"
    return _ollama_generate(prompt)

def compare_summaries(a: str, b: str, c: str) -> str:
    prompt = (
        "You are a concise analyst. Compare the three summaries provided.\n"
        "Output exactly this structure:\n"
        "Common themes:\n1. ...\n2. ...\n3. ...\n"
        "Key differences:\n1. ...\n2. ...\n3. ...\n"
        "Standout insight per source:\nA) ...\nB) ...\nC) ...\n"
        "Takeaway:\nOne sentence. Be crisp and non-redundant.\n\n"
        f"Summary A:\n{a}\n\nSummary B:\n{b}\n\nSummary C:\n{c}\n"
    )
    return _ollama_generate(prompt, num_predict=220, num_ctx=2048)


import re
from urllib.parse import urlparse
from datetime import datetime

def smart_filename_from_url(url: str, fallback: str = "summary") -> str:
    try:
        host = urlparse(url).netloc or ""
        host = re.sub(r"^www\.", "", host)
        host = re.sub(r"[^a-zA-Z0-9_-]+", "_", host).strip("_")
        date = datetime.now().date().isoformat()
        return f"{host or fallback}_{date}"
    except Exception:
        return fallback

def normalize_basename(name: str) -> str:
    name = (name or "").strip().lstrip(". ").strip()
    for ext in (".md", ".pdf", ".txt", ".markdown"):
        if name.lower().endswith(ext):
            name = name[: -len(ext)]
            break
    return name

def format_numbered_five(raw: str, url: str) -> str:
    title = f"### Here's the summary of {url}\n"
    text = (raw or "").strip()
    pts = re.split(r"(?m)^\s*(\d{1,2})\.\s+", text)
```

```
    numbered = []
    if len(pts) > 1:
        for i in range(1, len(pts) - 1, 2):
            body = pts[i+1].strip()
            if body:
                numbered.append(body)
        numbered = numbered[:5]
    if not numbered:
        lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
        cleaned = []
        for ln in lines:
            ln = re.sub(r"^\s*[-•\d]+\s*[.)-]?\s*", "", ln)
            if ln:
                cleaned.append(ln)
        numbered = cleaned[:5]
    while len(numbered) < 5:
        numbered.append("…")
    bullets = "\n".join(f"{i}. {p}" for i, p in enumerate(numbered, 1))
    return f"{title}\n{bullets}"
```

Tests

```
import types
from src import utils

def test_format_numbered_five_from_dashes():
    raw = "- a\n- b\n- c\n- d\n- e"
    out = utils.format_numbered_five(raw, "http://example.com")
    lines = [l for l in out.splitlines() if
l.strip().startswith(tuple(str(i)+'.' for i in range(1,6)))]
    assert len(lines) == 5

def test_smart_filename():
    fn = utils.smart_filename_from_url("https://www.site.com/foo")
    assert "site.com".replace(".","_").split("_")[0] in fn or "site" in fn

def test_normalize_basename():
    assert utils.normalize_basename("report.md") == "report"
    assert utils.normalize_basename("  .hidden  ") == ""
```